

Capitolo 53 — Operational Loop

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-53
Data master	2026-09-03
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/chapters/53_operational_loop.md
Source SHA	602ec99
Release	2026-09-04T09:45:54.479Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE IX — I loop fondamentali del WCM Stato: FROZEN **Data:** 2026-09-03 **Scope:** WCM generale, domain-agnostic

Nel capitolo precedente abbiamo visto la mappa dei nodi procedurali: richiesta, bootstrap, routing, execution, completion, consolidation e assurance. Quella mappa mostrava il territorio complessivo. Ora possiamo isolare uno dei circuiti che lo attraversano più spesso: l'**Operational Loop**.

Il nome può far pensare a un semplice ciclo «ricevi un compito, eseguilò, chiudilo». Nel WCM significa qualcosa di più preciso. L'Operational Loop è il circuito che permette a un lavoro di passare da **qualcosa che deve essere fatto** a **qualcosa che è stato realmente portato fino alla sua corretta condizione di arresto**, senza confondere la fine di una sessione con la fine del workflow e senza affidare alla sola memoria dell'AI la continuità operativa.

Un lavoro non è concluso perché è stato prodotto un output. È concluso quando il sistema ha raggiunto e verificato la vera condizione di stop prevista per quel lavoro.

53.1 Perché serve un loop operativo

Un sistema conversazionale tende naturalmente a lavorare per sessioni: riceve una richiesta, produce una risposta e termina il turno. Questo comportamento diventa fragile quando il lavoro richiede più passaggi, fonti, controlli o momenti di esecuzione.

Se un'attività di cinque passaggi viene interrotta dopo il terzo, ricostruirla soltanto dalla conversazione può produrre ripetizioni, salti o falsa closure. Il WCM tratta invece il lavoro come un workflow persistente: ciò che è già stato completato resta riconoscibile, la prossima transizione è identificabile e la condizione di stop non dipende dalla durata della sessione.

```
LAVORO DA FARE
↓
RICOSTRUISCI IL PUNTO REALE
↓
ESEGUI LA PROSSIMA TRANSIZIONE
↓
VERIFICA IL RISULTATO
↓
TRUE STOP RAGGIUNTA?
```

```
└─ NO → CONTINUA / CHECKPOINT / RESUME
└─ SÌ → CHIUSURA OPERATIVA
```

Il loop non promette che ogni attività possa essere automatizzata. Stabilisce come preservare continuità e verificabilità mentre il lavoro avanza.

53.2 Il punto di partenza non è sempre una nuova richiesta

L'Operational Loop può essere attivato da una richiesta nuova, ma anche da un evento, da un heartbeat, dalla disponibilità di una dipendenza o dalla ripresa di un workflow già iniziato.

Se ogni attivazione venisse trattata come un nuovo lavoro, il sistema potrebbe duplicare attività già eseguite o iniziare una nuova unità mentre quella precedente è incompleta. Per questo il WCM applica una logica di **Resume Priority**: quando esiste lavoro parziale e riprendibile nello scope pertinente, la continuità viene considerata prima dell'avvio di un nuovo lavoro equivalente.

```
ATTIVAZIONE
↓
ESISTE LAVORO RIPRENDIBILE?
└─ SÌ → RIPRENDI DAL CHECKPOINT
└─ NO → VALUTA NUOVO LAVORO
```

Resume Priority non riapre indiscriminatamente lo storico: vale per lavoro ancora operativo e pertinente secondo stato, authority e true stop condition.

53.3 Bootstrap: sapere abbastanza per continuare

Prima di agire, il sistema deve sapere dove si trova. Il bootstrap operativo recupera il contesto minimo sufficiente: ruolo, scope, authority, stato corrente, eventuale workflow attivo, ultima transizione completata, prossima transizione, processi e protocolli pertinenti, vincoli e vera condizione di stop.

Non è necessario caricare tutta la Persistent Organizational Memory. L'Operational Loop si appoggia a INDEX-FIRST e alla source precedence:

```
ENTRY POINT
↓
INDEX / MAPPA
↓
FONTI AUTOREVOLI NECESSARIE
↓
STATO E WORKFLOW PERTINENTI
↓
CONTESTO SUFFICIENTE
```

Il bootstrap termina quando il sistema sa abbastanza per eseguire correttamente la transizione successiva, non quando ha letto tutto ciò che potrebbe essere interessante.

53.4 Routing: dal significato al percorso applicabile

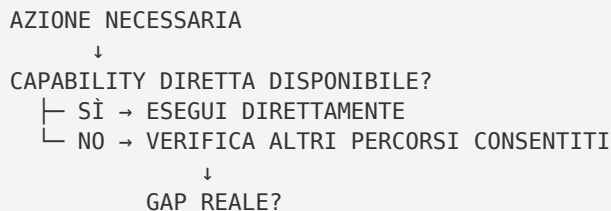
Una volta ricostruito il contesto, il WCM deve stabilire quale lavoro sta davvero eseguendo: goal, scope, authority, eventuale workflow da riprendere, processo principale, protocolli attivati, capability necessarie e gate applicabili.

Il routing evita due errori opposti. Il primo è l'**improvvisazione**, cioè scegliere ogni volta un percorso plausibile senza verificare quello previsto dal metodo. Il secondo è l'**iper-applicazione**, cioè eseguire tutte le regole possibili anche quando non sono pertinenti.

L'Operational Loop cerca invece il percorso minimo sufficiente ma governato.

53.5 Direct Before Delegate e capability reale

Una volta compreso cosa deve essere fatto, il WCM valuta come eseguirlo. Il principio Direct Before Delegate evita di introdurre servizi, agenti o passaggi aggiuntivi quando la capability necessaria è già disponibile direttamente.



Un ostacolo temporaneo non è automaticamente un capability gap. Prima di dichiarare che il lavoro non può proseguire, occorre verificare le capability disponibili e i percorsi consentiti. Questo non significa insistere indefinitamente: significa distinguere «non ho ancora trovato la strada» da «la strada non è disponibile».

53.6 Execution: fare il lavoro

Arrivati all'execution, il sistema deve produrre il risultato richiesto attraverso la capability appropriata. Un rischio tipico dei sistemi cognitivi è sostituire l'esecuzione con una spiegazione dell'esecuzione.

```
PIANO ≠ ESECUZIONE
PROPOSTA ≠ MUTAZIONE
OUTPUT DESCRITTIVO ≠ RISULTATO OPERATIVO
```

Allo stesso tempo, la capacità tecnica di effettuare una mutazione non costituisce da sola l'authority per effettuarla. L'Operational Loop deve quindi far avanzare il lavoro senza oltrepassare i confini di mandato.

53.7 Checkpoint: rendere il lavoro riprendibile

Dopo una transizione materiale, il sistema deve poter ricostruire ciò che è successo senza affidarsi alla memoria della sessione. Il checkpoint conserva gli execution facts necessari: cosa è stato completato, quale transizione viene dopo, quale stato operativo è valido e quale stop condition resta da raggiungere.

Il checkpoint non è il canone del significato. Non decide strategia, requisiti o authority. Serve a rispondere alla domanda «**a che punto è arrivata l'esecuzione?**», mentre canon e authority rispondono a «**che cosa significa il lavoro e chi può autorizzarlo?**». Questa separazione permette al runtime di rendere il lavoro riprendibile senza trasformarsi impropriamente in fonte di significato.

53.8 Contiguous Workflow Execution

L'Operational Loop non dovrebbe fermarsi a ogni micro-passaggio se la transizione successiva è già autorizzata, eseguibile e non incontra una vera stop condition.

```
STEP COMPLETATO
↓
ESISTE NEXT TRANSITION?
↓
È AUTORIZZATA E NELLO SCOPE?
↓
ESISTE UNA TRUE STOP QUI?
├ NO → CONTINUA
└ SÌ → STOP
```

Senza questa logica il sistema può produrre falsi stop: termina perché ha finito un turno, perché ha creato un file intermedio o perché chiede una nuova conferma quando l'authority esistente copre già la transizione successiva.

Contiguous Execution non elimina i gate. Rende più chiaro quali stop siano reali: authority mancante, failure, dipendenza non risolta, condizione contrattuale non soddisfatta o true stop esplicita.

53.9 Completion Gate: aver prodotto qualcosa non significa aver finito

Dopo l'esecuzione arriva la domanda decisiva: **abbiamo davvero finito?**

Il Completion Gate verifica che gli output richiesti esistano, che la true stop condition sia stata raggiunta, che il checkpoint sia corrente e che non restino requisiti obbligatori alla closure. Se manca una condizione necessaria, **COMPLETED** non è lo stato corretto.

Il lavoro può essere ancora in esecuzione, in attesa di authority, bloccato da una condizione reale, interrotto ma riprendibile, fallito con evidence oppure completato. Questa distinzione evita di usare «finito» come contenitore generico per situazioni diverse.

53.10 Interrupted Resumable: fermarsi senza perdere il lavoro

Non tutti gli stop sono errori e non tutte le interruzioni sono closure. Se un limite tecnico reale o una dipendenza impedisce di continuare, ma il lavoro già svolto è valido e il workflow può essere ripreso, il sistema preserva un checkpoint coerente e rende esplicita la necessità di resume.

```

IMPOSSIBILE CONTINUARE ORA
↓
LAVORO PARZIALE VALIDO?
├ NO → FAILURE / EVIDENCE
└ SÌ → CHECKPOINT
    ↓
    INTERRUPTED_RESUMABLE
    ↓
    FUTURE RESUME

```

È molto diverso da dichiarare **COMPLETED**: l'attività incompleta non scompare soltanto perché una run è terminata.

53.11 Il loop attraverso più sessioni

L'Operational Loop non coincide con una singola sessione, un singolo agente o un singolo heartbeat. Può attraversare più attivazioni mantenendo lo stesso workflow:

```

RUN A
bootstrap → step 1 → step 2 → checkpoint

RUN B
resume → step 3 → gate → checkpoint

RUN C
resume → step 4 → completion gate → true stop

```

Ciò che rende queste run un unico percorso non è la continuità della conversazione. È la continuità dello stato persistente, dell'autorità e della true stop condition. Questo riduce la dipendenza dal contesto volatile della singola sessione.

53.12 L'esperienza operativa entra nel loop

La Method Experience Memory contiene un precedente direttamente comparabile: in un workflow editoriale sequenziale è stato osservato con evidenza positiva un pattern composto da perimetro finito, una sola unità per run, selezione del primo elemento non completato, Resume Priority e stato reale persistito fuori dalla sessione.

```

PERIMETRO FINITO
↓
UNA UNITÀ PER RUN
↓
LEGGI STATO REALE
↓
RIPRENDI IL PARZIALE SE ESISTE
↓
COMPLETA FINO ALLA TRUE STOP
↓
LA RUN NON GOVERNA IL PROPRIO SCHEDULER

```

L'evidenza è circoscritta: non dimostra che questa sia la soluzione migliore per processi event-driven, code dinamiche o workload senza perimetro finito. Mostra però un comportamento riutilizzabile quando il contesto è comparabile.

La stessa memoria conserva anche una failure complementare: una configurazione con worker ricorrente senza limite e Guard separato è stata osservata in uno stato di disabilitazione prematura prima del completamento del perimetro. La causa tecnica esatta non è stata dimostrata e non va inventata.

La lezione non è «worker + Guard è sempre sbagliato». È più rigorosa: **quando esiste un pattern più semplice con evidenza positiva nello stesso tipo di lavoro, non conviene reinventare un meccanismo più complesso senza un vincolo nuovo che lo renda necessario.**

53.13 Authority: il loop non crea permesso

Un workflow persistente può dire quale transizione viene dopo. Non può inventare l'authority necessaria per attraversarla.

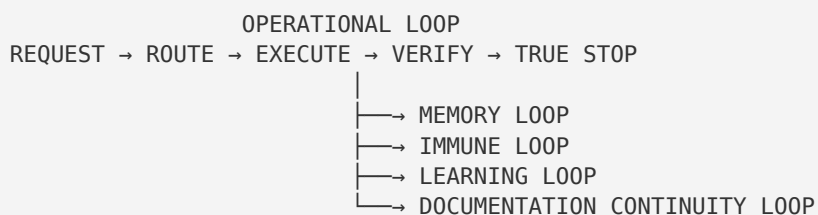
```
NEXT TRANSITION ESISTE
    ≠
NEXT TRANSITION È AUTORIZZATA
```

Quando un gate richiede una decisione riservata all'authority competente, lo stop è corretto. Non è una failure del loop. Continuità operativa significa avanzare finché le condizioni previste lo consentono e fermarsi esattamente quando una condizione reale lo richiede.

53.14 Il rapporto con gli altri loop

L'Operational Loop porta avanti il lavoro, ma non sostituisce i circuiti che governano ciò che il lavoro produce.

Se l'esecuzione genera un delta materiale, può attivarsi il **Memory Loop**. Se occorre verificare l'integrità della conoscenza persistente, interviene l'**Immune Loop**. Se dall'esperienza emerge un pattern o una failure utile, entra in gioco il **Learning Loop**. Se una modifica materiale impatta documentazione corrente, può attivarsi il **Documentation Continuity Loop**.



Le frecce non significano che tutti i loop si attivino sempre. Dipende dai trigger reali.

53.15 Cognizione e determinismo nel loop

L'Operational Loop contiene sia problemi di significato sia problemi meccanici. Comprendere una richiesta, interpretare una fonte o valutare una contraddizione semantica richiede capacità cognitive. Verificare identificatori, contratti, checkpoint strutturati o proiezioni di stato può invece essere affidato, quando formalizzato, a

componenti deterministici.

Il principio non è togliere l'AI dal loop. È usare la cognizione dove serve e non usarla come sostituto di controlli meccanici che possono essere eseguiti in modo ripetibile.

53.16 Che cosa rende sano l'Operational Loop

Un Operational Loop sano riprende dal punto reale invece di ricominciare; recupera il contesto selettivamente; mantiene authority e stato di esecuzione separati; verifica le capability prima di dichiarare un blocco; continua gli step già autorizzati; lascia checkpoint sufficienti; non scambia la fine della sessione per la fine del workflow; usa **COMPLETED** soltanto quando la true stop condition è soddisfatta.

Segnali di fragilità sono invece lavoro già eseguito che viene ripetuto, nuove unità avviate mentre esiste lavoro parziale equivalente, stop dopo ogni micro-passaggio senza gate reale, impossibilità dichiarata senza capability check, output prodotto senza closure verificata, stato ricostruito soltanto dalla conversazione o complessità aggiunta ignorando precedenti operativi pertinenti.

53.17 Dall'Operational Loop al Memory Loop

L'Operational Loop risponde alla domanda:

come porto avanti il lavoro fino alla sua vera condizione di stop senza perdere continuità?

Una volta eseguito il lavoro nasce però una seconda domanda:

che cosa di ciò che è successo deve sopravvivere, dove deve essere consolidato e come evitiamo che la memoria persistente diventi incoerente?

Questa domanda apre il capitolo successivo: il **Memory Loop**.

In sintesi

L'Operational Loop è il circuito di continuità dell'esecuzione WCM. Parte da una richiesta, un evento o un workflow da riprendere; ricostruisce il contesto minimo sufficiente; determina il percorso applicabile; verifica capability e authority; esegue; aggiorna i checkpoint; continua attraverso le transizioni già autorizzate; distingue un'interruzione riprendibile da una vera closure; e termina soltanto quando la true stop condition è stata realmente raggiunta.

Il suo valore non sta nell'automatizzare qualsiasi cosa. Sta nel rendere il lavoro **riprendibile, governato e verificabile oltre i confini della singola sessione**.

La fine di una risposta non è la fine del lavoro. Nel WCM, il lavoro finisce quando il workflow può dimostrarlo.